TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE OAXACA

MATERIA:

LENGUAJES Y AUTÓMATAS II

CLAVE DE MATERIA:

SCD1016

CARRERA:

INGENIERÍA EN SISTEMAS COMPUTACIONALES

PROYECTO INTEGRADOR – FASE 2: DISEÑO DEL COMPILADOR

EmoLang

ALUMNOS:

GARCIA GABRIEL BETSAIDA
GINER COACHE NIERIKA
JIMENEZ LOPEZ FERNANDO ARATH
LEÓN RAMÍREZ CARLOS PAVEL
ORTIZ MARTÍNEZ JOSÉ JULIÁN

NOMBRE DEL PROFESOR:

VELAZQUEZ HERNANDEZ MARICARMEN MONTSERRAT

GRUPO: 7SB

OAXACA DE JUÁREZ, OAXACA
NOVIEMBRE DE 2026

Índice

1. Alcance del prototipo **3**
1.1 Qué incluye y qué no . . . 3
1.2 Tipos de datos . . . 3
1.3 Estructuras de control . . . 4
1.4 Métricas y acciones . . . 4
2. Gramática libre de contexto (GLC) **5**
2.1 Componentes léxicos (T) . . . 5
2.2 Producciones (P) . . . 6
2.3 Programa de ejemplo . . . 6
3. Reglas semánticas dirigidas por la sintaxis **8**
3.1 Atributos . . . 8
3.2 Esquema de traducción . . . 9
3.3 Reglas de tipos . . . 10
3.4 Verificaciones semánticas . . . 10
4. Construcción de árboles de expresión **11**
5. Pila semántica **11**
5.1 Funcionamiento . . . 12
5.2 Ejemplo con la línea 24 . . . 12
6. Tabla de símbolos **12**
7. Cuádruplas **13**
7.1 Bloque init: variables (líneas 6 a 16) . . . 14
7.2 Bloque gestos (líneas 21 a 29) . . . 14
7.3 Bloque al_entrar del estado alerta (líneas 74 a 79) . . . 16
7.4 Bloques mientras (líneas 45 y 59) . . . 16
7.5 Un ciclo repetir . . . 17
7.6 Relación con los autómatas . . . 18
8. Técnicas de optimización **19**
8.1 Técnicas . . . 20
8.2 Resultado por bloque . . . 20
8.3 Bloque gestos optimizado . . . 21
8.4 Bloque al_entrar de alerta optimizado . . . 22
8.5 Ciclo repetir desenrollado . . . 22
8.6 Comprobación . . . 23
9. Plataforma y arquitectura **23**
9.1 Elección de la plataforma . . . 23
9.2 Arquitectura . . . 24
9.3 Tecnologías . . . 24
Referencias **25**

1. Alcance del prototipo

El prototipo compila un programa EmoLang (extensión `.emo`) a un archivo `.emoc` y lo ejecuta en tiempo real, en CPU, con un solo rostro frente a una cámara RGB. Cumple el objetivo del anteproyecto: los gestos y los estados se traducen a autómatas finitos, y las condiciones y acciones a cuádruplas.

1.1 Qué incluye y qué no

Incluido en el prototipo	**Fuera del prototipo**
Uno o varios dominios por archivo; se ejecuta uno a la vez	Importar o combinar varios archivos
Un rostro frente a una cámara RGB	Varios rostros o varias cámaras
Módulos de dominio: conducción y rehabilitación	Otros módulos con acciones propias
14 métricas faciales predefinidas (1.4)	Métricas nuevas o modelos de visión propios
Funciones predefinidas (1.4)	Funciones definidas por el usuario
Ejecución en CPU; el ritmo de la cámara se normaliza al fps del programa	GPU, celular o hardware embebido
Mensajes de error con línea y columna; varios errores por compilación	Corrección automática de errores

1.2 Tipos de datos

Tipo	**Ejemplo**	**Descripción**
`int`	`30`	Número entero
`decimal`	`0.5`	Número con punto decimal
`bool`	`verdadero, falso`	Valor lógico
`string`	`"Mire al frente"`	Texto
`duracion`	`500 ms, 1.5 s`	Tiempo; se guarda en milisegundos al compilar
`proporcion`	`0.21, 60%`	Valor entre 0 y 1; 60% se guarda como 0.6
`gesto`	`gesto bostezo = mar > umbral durante 1.5 s;`	Condición sostenida en el tiempo; se compila a un AFD
`secuencia`	`secuencia fatiga = [bostezo, cabeceo] en 10 s;`	Hasta 5 gestos en orden, dentro de una ventana; se compila a un AFD

Conversiones implícitas: `int` a `decimal`, `decimal` a `proporcion` y viceversa (si el valor es constante se verifica que esté entre 0 y 1). `1 s` equivale a `1000 ms`. No se permite `decimal` a `int` ni un número sin unidad como duración.

1.3 Estructuras de control

Estructura	Sintaxis	Uso
Condicional	`si c { … } sino { … }`	Ejecuta un bloque u otro según la condición
Ciclo con número fijo	`repetir N veces { … }`	Repite el bloque N veces dentro del cuadro
Ciclo por cuadro	`mientras c { … }`	Se declara en un estado; se revisa una vez por cuadro y no bloquea
Máquina de estados	`maquina, inicial, estado`	Define los estados y el estado de inicio
Transiciones	`cuando g ir_a E; ir_a E; cuando tras D ir_a E;`	Cambio de estado por un evento, desde un bloque o por tiempo
Bloques de estado	`al_entrar { … } cuando g { … }`	Lo que se hace al entrar y al ocurrir un evento
Evento sin efecto	`ignorar g;`	Declara que el estado no reacciona a g a propósito

1.4 Métricas y acciones

Métrica	Tipo	Qué mide
`ear`	`proporcion`	Razón de aspecto del ojo (promedio). Ojo abierto ~0.25-0.35, cerrado < 0.2
`ear_izq`	`proporcion`	EAR del ojo izquierdo
`ear_der`	`proporcion`	EAR del ojo derecho
`mar`	`proporcion`	Razón de aspecto de la boca. Boca cerrada ~0.0-0.1, bostezo > 0.6
`sonrisa`	`proporcion`	Intensidad de sonrisa normalizada (0 neutro, 1 máxima)
`sonrisa_izq`	`proporcion`	Elevación de la comisura izquierda
`sonrisa_der`	`proporcion`	Elevación de la comisura derecha
`cejas`	`proporcion`	Elevación de cejas normalizada
`simetria`	`proporcion`	Simetría facial izquierda/derecha (1 = perfectamente simétrico)
`perclos`	`proporcion`	Proporción de tiempo con ojos cerrados en los últimos 60 s
`pitch`	`decimal`	Inclinación vertical de la cabeza en grados (positivo = hacia abajo)
`yaw`	`decimal`	Giro horizontal de la cabeza en grados
`roll`	`decimal`	Inclinación lateral de la cabeza en grados
`rostro`	`bool`	Verdadero si hay un rostro detectado en el cuadro

La métrica `ear` sigue a Soukupová y Čech (2016) y `perclos` a Wierwille et al. (1994). El dominio de una función se revisa al compilar, según el nombre del dominio.

Función	Dominio	Qué hace
`alarma(m), detener_alarma()`	conducción	Activa la alarma con el mensaje m; la apaga
`contar_repeticion(), repeticiones()`	rehabilitación	Cuenta una repetición con su simetría; devuelve cuántas van
`registrar(m)`	ambos	Agrega m a la bitácora, que se exporta a CSV
`mostrar(a, b, …)`	ambos	Muestra uno o más valores
`tiempo_en_estado()`	ambos	Tiempo desde que se entró al estado actual
`abs, a_porcentaje, a_proporcion, a_ms, a_s`	ambos	Valor absoluto y conversiones explícitas

2. Gramática libre de contexto (GLC)

EmoLang se define con la gramática G = (N, T, P, S), donde S = <programa>, N son los símbolos entre < > y T son los componentes léxicos de 2.1. El analizador léxico es un AFD escrito a mano que aplica la regla del lexema más largo.

2.1 Componentes léxicos (T)

Token	Patrón	Ejemplos
Palabras reservadas	Lista fija	`dominio sensor usar var gesto secuencia maquina inicial estado al_entrar cuando ignorar ir_a tras durante hasta tolerando en si sino mientras repetir veces int decimal bool string duracion proporcion verdadero falso y o no ms s`
ID	Letra o _, seguida de letras, dígitos o _	`umbral_ojo, alertas, ear`
ENTERO	Dígitos	`30`
DECIMAL	Dígitos, punto y dígitos	`0.21`
Unidades	ms, s y %, juntas o separadas del número	`500ms, 1.5 s, 60%`
CADENA	Texto entre comillas dobles, en una línea	`"¡Despierte!"`
Comparación	<, >, <=, >=, ==, !=	`ear < 0.2`
Operadores	+, -, *, /, =	`alertas + 1`
Delimitadores	{ } ( ) [ ] , ; :	
Comentarios (se descartan)	// hasta fin de línea y /* bloque */	`// umbrales`

Las métricas y las funciones no son palabras reservadas: el analizador léxico las entrega como `ID` y el semántico las resuelve en la tabla de símbolos. Por eso una variable no puede llamarse como una métrica.

2.2 Producciones (P)

```
 1 <programa>    → <dominio>+
 2 <dominio>     → dominio ID { <miembro>* }
 3 <miembro>     → <sensor> | <decl> | <gesto> | <secuencia> | <maquina>
 4 <sensor>      → sensor ID { ( usar <ids> ; | <decl> )* }
 5 <ids>         → ID ( , ID )*
 6 <decl>        → var ID : <tipo> = <expr> ;
 7 <tipo>        → int | decimal | bool | string | duracion | proporcion | gesto | secuencia
 8 <gesto>       → gesto ID = <expr> durante <expr> ( hasta <expr> )? ( tolerando <expr> )? ;
 9 <secuencia>   → secuencia ID = [ <ids> ] ( en <expr> )? ;
10 <maquina>     → maquina ID inicial ID { <estado>+ }
11 <estado>      → estado ID { <miembro_e>* }
12 <miembro_e>   → <decl>
                 | al_entrar <bloque>
                 | cuando ID ir_a ID ;
                 | cuando ID <bloque>
                 | cuando tras <expr> ir_a ID ;
                 | mientras <expr> <bloque>
                 | ignorar <ids> ;
13 <bloque>      → { <sentencia>* }
14 <sentencia>   → <decl> | ID = <expr> ; | <si> | repetir <expr> veces <bloque>
                 | ir_a ID ; | <llamada> ;
15 <si>          → si <expr> <bloque> ( sino ( <si> | <bloque> ) )?
16 <expr>        → <conj> ( o <conj> )*
17 <conj>        → <neg> ( y <neg> )*
18 <neg>         → no <neg> | <rel>
19 <rel>         → <suma> ( ( < | <= | > | >= | == | != ) <suma> )?
20 <suma>        → <term> ( ( + | - ) <term> )*
21 <term>        → <unario> ( ( * | / ) <unario> )*
22 <unario>      → - <unario> | <primario>
23 <primario>    → ( ENTERO | DECIMAL ) ( ms | s | % )? | CADENA | verdadero | falso
                 | ID | <llamada> | ( <expr> )
24 <llamada>     → ID ( ( <expr> ( , <expr> )* )? )
```

`X*` significa cero o más veces, `X+` una o más y `X?` opcional. Son abreviaturas: por ejemplo, `<decl>*` equivale a `<decls> → <decl> <decls> | ε`, así que la gramática sigue siendo libre de contexto. Las producciones 16 a 22 dan la precedencia, de mayor a menor: signo, luego * y /, + y -, las comparaciones, `no`, `y` y al final `o`. La recursión izquierda se escribe como repetición porque el analizador es descendente recursivo. La gramática es LL(1), salvo en `<sentencia>` con `ID`: un token más decide entre asignación y llamada. `gesto` y `secuencia` como tipo de una variable se aceptan en la sintaxis y los rechaza el análisis semántico, con un mensaje que indica cómo declararlos.

2.3 Programa de ejemplo

Todos los ejemplos del documento usan este programa, que es el archivo `ejemplos/conduccion.emo` del proyecto. Los umbrales se ajustaron con las pruebas de cámara del equipo.

Listado 1. Detección de fatiga (dominio conducción).

```
01 // EmoLang. Módulo de transporte: detección de fatiga.
02 dominio conduccion {
03
04     sensor camara {
05         usar ear, mar, pitch, yaw, perclos;
06         var fps : int = 30;
07     }
08
09     // Umbrales configurables por el supervisor de transporte
10     var umbral_ojo     : proporcion = 0.20;
11     // Valores ajustados con el diagnóstico de cámara (python -m emolang diagnostico)
12     var umbral_bostezo : proporcion = 33%;
13     var umbral_perclos : proporcion = 15%;
14     var cabeceo_grados : decimal    = 10.0;
15     var giro_grados    : decimal    = 20.0;
16     var alertas        : int        = 0;
17
18     // Gestos: condiciones sostenidas en el tiempo.
19     // 'tolerando' perdona interrupciones breves (ruido o cuadros perdidos)
20     // sin reiniciar el conteo de la duración.
21     gesto parpadeo     = ear < umbral_ojo y mar < umbral_bostezo durante 60 ms hasta 400 ms;
22     gesto microsueno   = ear < umbral_ojo y mar < umbral_bostezo durante 1 s tolerando 100 ms;
23     gesto bostezo      = mar > umbral_bostezo durante 1.5 s tolerando 200 ms;
24     gesto cabeceo      = pitch > cabeceo_grados y ear < 0.27 durante 300 ms tolerando 100 ms;
25     gesto distraccion  = yaw > giro_grados o yaw < -giro_grados durante 2 s tolerando 200 ms;
26     gesto atento       = pitch < 10.0 y yaw < 20.0 y yaw > -20.0 durante 3 s tolerando 200 ms;
27
28     // Patrón de fatiga: bostezo seguido de cabeceo en menos de 10 s
29     secuencia fatiga = [bostezo, cabeceo] en 10 s;
30
31     // Máquina de estados del monitor.
32     maquina monitor inicial normal {
33
34         estado normal {
35             var parpadeos : int = 0;
36             al_entrar { detener_alarma(); }
37
38             cuando parpadeo   { parpadeos = parpadeos + 1; }
39             cuando bostezo    { registrar("bostezo detectado"); ir_a somnoliento; }
40             cuando microsueno ir_a alerta;
41             cuando distraccion ir_a distraido;
42             cuando fatiga     ir_a alerta;
43             ignorar atento;                     // estar atento ya es lo normal
44
45             mientras perclos > umbral_perclos {
46                 registrar("PERCLOS alto: " + perclos);
47                 ir_a somnoliento;
48             }
49         }
50
51         estado somnoliento {
52             al_entrar { mostrar("Conductor somnoliento, vigilando..."); }
53             cuando microsueno ir_a alerta;
54             cuando fatiga     ir_a alerta;
55             cuando distraccion ir_a distraido;   // somnoliento y sin mirar al frente
56             cuando cabeceo    { registrar("cabeceo"); }
57             ignorar bostezo, atento;             // ya se sabe que está somnoliento
58             // solo se relaja cuando pasaron 15 s Y el PERCLOS ya bajó
59             mientras tiempo_en_estado() > 15 s y perclos < umbral_perclos {
60                 ir_a normal;
61             }
62         }
63
```

```
64          estado distraido {
65              al_entrar { alarma("Mire al frente"); }
66              cuando atento ir_a normal;
67              cuando microsueno ir_a alerta;
68              cuando tras 30 s ir_a alerta;         // lleva demasiado sin mirar al frente
69              ignorar bostezo, distraccion, fatiga; // la alarma ya está sonando
70          }
71
72          estado alerta {
73              al_entrar {
74                  alertas = alertas + 1;
75                  alarma("¡Despierte! Riesgo de microsueño");
76                  registrar("alerta #" + alertas);
77                  si alertas >= 3 {
78                      registrar("Se recomienda detener el vehículo");
79                  }
80              }
81              cuando atento ir_a normal;
82              ignorar bostezo, microsueno, distraccion, fatiga;  // alarma activa hasta estar atento
83          }
84      }
85  }
```

3. Reglas semánticas dirigidas por la sintaxis

El analizador sintáctico construye el árbol de sintaxis abstracta (AST). Después, dos recorridos aplican las reglas: el análisis semántico revisa tipos y nombres; el generador de código intermedio emite las cuádruplas. Se separan porque los nombres de los estados, gestos y secuencias pueden usarse antes de declararse.

3.1 Atributos

Atributo	Significado
`.tipo`	Tipo del resultado (int, decimal, bool, …). Si hubo un error, queda vacío para no repetir mensajes
`.lugar`	Dónde queda el valor: una variable, una constante o un temporal (t1, t2, …)
`.valor`	Valor de una constante ya convertido: milisegundos o proporción de 0 a 1
`.linea`, `.columna`	Posición en el archivo, para los mensajes de error

3.2 Esquema de traducción

Producción	Regla semántica
<decl> → var ID : <tipo> = <expr> ;	ID no debe existir en el ámbito actual y el tipo de <expr> debe ser asignable a <tipo>. Se inserta ID en la tabla de símbolos y se emite (=, <expr>.lugar, –, ID) en el bloque init o al_entrar que corresponda
<sentencia> → ID = <expr> ;	ID debe existir y ser una variable; una métrica no se puede asignar. Se emite (=, <expr>.lugar, –, ID)
<suma> → <suma>₁ + <term>	El tipo se calcula con las reglas de 3.3. Se crea un temporal t y se emite (+, <suma>.lugar, <term>.lugar, t). Si un operando es string, se emite texto y concat
<term> → <term>₁ * <unario>	Igual que la suma, con *
<rel> → <suma>₁ OP <suma>₂	Los dos lados deben poder compararse. El resultado es bool y se emite (OP, <suma>₁.lugar, <suma>₂.lugar, t)
<primario> → ID	Se busca ID en la tabla; su tipo y su lugar pasan a la pila. Si es una métrica, debe estar en usar
<primario> → NUM ms | NUM s	El valor se guarda en milisegundos desde el análisis sintáctico; se usa como constante
<primario> → NUM %	El valor se guarda como n / 100 y no puede pasar de 1
<gesto> → gesto ID = <expr> durante …	<expr> debe ser bool y sin llamadas; las duraciones, constantes. Se construye el AFD del gesto y se emite (afd, <expr>.lugar, –, ID) en el bloque gestos
<secuencia> → secuencia ID = [<ids>] …	Cada nombre debe ser un gesto, de 1 a 5. Con uno solo se advierte. Se construye el AFD (KMP) y se emite (sec, –, –, ID)
<si> → si <expr> <bloque> sino <bloque>	<expr> debe ser bool. Se emite un gotoF hacia el sino y un goto hacia el final
<miembro_e> → mientras <expr> <bloque>	<expr> debe ser bool. Crea un bloque propio; se emite un gotoF hacia su final. No hay salto de regreso: se revisa una vez por cuadro
<sentencia> → repetir <expr> veces <bloque>	<expr> debe ser int. Se emite un contador, su comparación y un salto de regreso. Si no es constante, además se emite chk_rep
<llamada> → ID (…)	La función debe existir, recibir los tipos que pide y estar permitida en el dominio. Se emite un param por argumento y (call, ID, n, t)
<estado> → estado ID { … }	Se abre su ámbito; sus variables se reinician cada vez que se entra al estado
<maquina> → maquina ID inicial ID { … }	El estado inicial debe existir. Los estados se insertan antes de revisarlos, porque un ir_a puede nombrar uno declarado después
<miembro_e> → cuando ID …	ID debe ser un gesto o secuencia y el destino un estado de la misma máquina
<miembro_e> → cuando tras <expr> ir_a ID ;	La duración debe ser constante y mayor que cero. Se agrega a la tabla de despacho del estado

3.3 Reglas de tipos

Operación	Resultado
int + int, int - int, int * int	int
int + decimal, decimal + proporcion y demás mezclas numéricas	decimal
número / número	decimal
duracion + duracion, duracion - duracion	duracion
duracion * número, duracion / número	duracion
duracion / duracion	decimal
string + cualquier tipo	string
Comparación entre números o entre duraciones	bool
== y != entre dos string o dos bool	bool
y, o, no	Solo con bool; el resultado es bool
Cualquier otra combinación	Error de tipo

3.4 Verificaciones semánticas

Los mensajes de esta tabla son los que produce el compilador con cada ejemplo.

Verificación	Ejemplo	Mensaje del compilador
Variable no declarada	`alertas = velocidad;`	identificador 'velocidad' no declarado
Declarada dos veces en el mismo ámbito	`dos var alertas`	'alertas' ya fue declarado en este ámbito (línea 1)
Tipos incompatibles	`alertas = "hola";`	no se puede asignar string a 'alertas' de tipo int
Asignar a una métrica	`ear = 0.5;`	'ear' es metrica y no se puede asignar
Métrica no habilitada en usar	`sonrisa con usar ear, mar`	la métrica 'sonrisa' no está habilitada en 'usar' del sensor
Umbral fuera de rango	`mar > 120%`	umbral fuera de rango: una proporcion debe estar entre 0 y 1 (0% y 100%), se obtuvo 1.2
Gesto imposible	`ear < 0.2 y ear > 0.3`	gesto imposible: la condición de 'g' nunca puede cumplirse
Duración sin unidad	`durante 5`	'durante' requiere una duración con unidad (ms o s), se recibió int
Secuencia demasiado larga	`seis gestos`	la secuencia 's1' tiene 6 gestos; el máximo es 5
Transiciones contradictorias	`dos cuando g a destinos distintos`	transiciones contradictorias en 'a': el evento 'g' lleva a b, c
Estado inalcanzable	`b sin transición que llegue`	estado inalcanzable: 'b' no puede alcanzarse desde el estado inicial 'a'
Estado inicial inexistente	`inicial z`	el estado inicial 'z' no existe en la máquina 'm'
Acción fuera de su dominio	`alarma en rehabilitacion`	'alarma' solo está disponible en el dominio conduccion

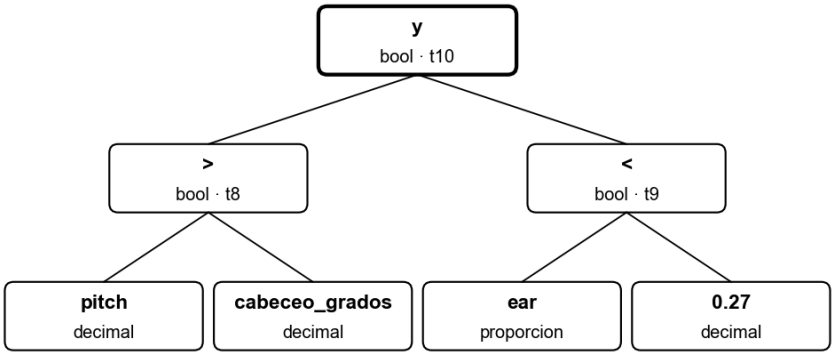
Además, el compilador emite advertencias que no impiden compilar: umbrales atípicos, gestos sin usar, condiciones siempre verdaderas y eventos atendidos en unos estados pero olvidados en otros sin declararlos con `ignorar`. Para detectar gestos imposibles, la condición se lleva a forma normal disyuntiva sobre intervalos por métrica y se interseca con el rango válido de cada una. Los estados inalcanzables se detectan con un recorrido en anchura desde el estado inicial.

4. Construcción de árboles de expresión

El analizador sintáctico arma un árbol por cada expresión: primero las hojas y luego cada operador con sus hijos. El análisis semántico recorre el árbol y le asigna a cada nodo su tipo con las reglas de 3.3. Ejemplo con la condición de la línea 24 del Listado 1, `pitch > cabeceo_grados y ear < 0.27`:

1. `hoja(pitch)`: decimal, tomado de la tabla de símbolos
2. `hoja(cabeceo_grados)`: decimal, tomado de la tabla de símbolos
3. `nodo(>, pitch, cabeceo_grados)`: bool, porque es una comparación entre números
4. `hoja(ear)`: proporcion, tomado de la tabla de símbolos
5. `hoja(0.27)`: decimal
6. `nodo(<, ear, 0.27)`: bool, porque es una comparación entre números
7. `nodo(y, >, <)`: bool, porque los dos operandos son bool

Figura 1. Árbol de expresión de la línea 24



Las comparaciones quedan más abajo que `y`, por eso se calculan primero. Al recorrer el árbol en posorden (`pitch cabeceo_grados > ear 0.27 < y`) salen las cuádruplas `(>, pitch, cabeceo_grados, t8)`, `(<, ear, 0.27, t9)` y `(y, t8, t9, t10)` del bloque gestos (sección 7.2).

5. Pila semántica

La pila semántica guarda el lugar y el tipo de cada parte ya reconocida de una expresión. La usa el generador de código intermedio (`emolang/intermedio.py`) mientras recorre el árbol en posorden.

5.1 Funcionamiento

1. Al llegar a una hoja, mete en la pila su lugar y su tipo.
2. Al llegar a un operador, saca sus operandos, emite la cuádrupla con el tipo que ya calculó el análisis semántico y mete el temporal del resultado.
3. Al terminar la expresión, la pila queda con un solo elemento: el lugar del resultado. Lo saca quien la pidió (una asignación, un gesto, un si).

Los errores de tipo se detectan antes, en el análisis semántico: el nodo con error queda sin tipo para no repetir el mensaje, y si hay errores no se genera código.

5.2 Ejemplo con la línea 24

La tabla es la traza que registra el generador para `pitch > cabeceo_grados y ear < 0.27`. Cada entrada de la pila es (lugar, tipo) y el tope está a la derecha.

Paso	Qué hace el generador	Pila semántica	Cuádrupla emitida
1	Lee pitch	`[(pitch, decimal)]`	
2	Lee cabeceo_grados	`[(pitch, decimal), (cabeceo_grados, decimal)]`	
3	Reconoce pitch > cabeceo_grados: decimal > decimal = bool	`[(t8, bool)]`	`(>, pitch, cabeceo_grados, t8)`
4	Lee ear	`[(t8, bool), (ear, proporcion)]`	
5	Lee la constante 0.27	`[(t8, bool), (ear, proporcion), (0.27, decimal)]`	
6	Reconoce ear < 0.27: proporcion < decimal = bool	`[(t8, bool), (t9, bool)]`	`(<, ear, 0.27, t9)`
7	Reconoce t8 y t9: bool y bool = bool	`[(t10, bool)]`	`(y, t8, t9, t10)`
8	Reconoce el gesto cabeceo: la condición es bool	`[]`	`(afd, t10, –, cabeceo)`

6. Tabla de símbolos

La tabla guarda cada nombre declarado. Los ámbitos, del más externo al más interno, son: **global** (métricas y funciones), **dominio** (variables del dominio y de sus sensores, gestos, secuencias y máquinas), **máquina** (estados), **estado** (variables locales) y **bloque** (variables declaradas dentro de llaves). Un nombre se busca del ámbito actual hacia afuera. Las variables de un sensor se guardan en el ámbito del dominio con la etiqueta de su sensor. Tabla final del Listado 1:

Nombre	Categoría	Tipo	Ámbito	Valor o dato	Línea
`fps`	variable	int	sensor camara	30	6
`umbral_ojo`	variable	proporcion	dominio conduccion	0.2	10
`umbral_bostezo`	variable	proporcion	dominio conduccion	0.33	12

Nombre	Categoría	Tipo	Ámbito	Valor o dato	Línea
umbral_perclos	variable	proporcion	dominio conduccion	0.15	13
cabeceo_grados	variable	decimal	dominio conduccion	10	14
giro_grados	variable	decimal	dominio conduccion	20	15
alertas	variable	int	dominio conduccion	0	16
parpadeo	gesto	gesto	dominio conduccion	60 ms a 400 ms	21
microsueno	gesto	gesto	dominio conduccion	1000 ms; tolera 100 ms	22
bostezo	gesto	gesto	dominio conduccion	1500 ms; tolera 200 ms	23
cabeceo	gesto	gesto	dominio conduccion	300 ms; tolera 100 ms	24
distraccion	gesto	gesto	dominio conduccion	2000 ms; tolera 200 ms	25
atento	gesto	gesto	dominio conduccion	3000 ms; tolera 200 ms	26
fatiga	secuencia	secuencia	dominio conduccion	bostezo, cabeceo; 10000 ms	29
monitor	maquina	maquina	dominio conduccion	inicial: normal	32
normal	estado	estado	maquina monitor		34
somnoliento	estado	estado	maquina monitor		51
distraido	estado	estado	maquina monitor		64
alerta	estado	estado	maquina monitor		72
parpadeos	variable	int	estado normal	0	35
ear	métrica	proporcion	global	0 a 1; habilitada en usar	5
mar	métrica	proporcion	global	0 a 1; habilitada en usar	5
perclos	métrica	proporcion	global	0 a 1; habilitada en usar	5
pitch	métrica	decimal	global	-90 a 90; habilitada en usar	5
yaw	métrica	decimal	global	-90 a 90; habilitada en usar	5

7. Cuádruplas

Cada cuádrupla tiene la forma (operador, arg1, arg2, resultado). Las genera el compilador por bloques: `init` con las variables, `gestos` con las condiciones, y por cada estado sus bloques `al_entrar`, `cuando` y `mientras`. Los temporales (`t1, t2, …`) vuelven a empezar en cada bloque. Una variable local se escribe con su estado, como `normal.parpadeos`. Las duraciones y los porcentajes ya llegan convertidos: el analizador sintáctico guarda `1.5 s` como `1500` y `33%` como `0.33`. Para ver las de cualquier programa: `python -m emolang cuadruplas archivo.emo --sin-optimizar`.

7.1 Bloque init: variables (líneas 6 a 16)

#	Operador	Arg1	Arg2	Resultado	Qué se hace
1	`=`	`30`	–	`fps`	Guarda 30 en fps
2	`=`	`0.2`	–	`umbral_ojo`	Guarda 0.2 en umbral_ojo
3	`=`	`0.33`	–	`umbral_bostezo`	Guarda 0.33 en umbral_bostezo
4	`=`	`0.15`	–	`umbral_perclos`	Guarda 0.15 en umbral_perclos
5	`=`	`10`	–	`cabeceo_grados`	Guarda 10 en cabeceo_grados
6	`=`	`20`	–	`giro_grados`	Guarda 20 en giro_grados
7	`=`	`0`	–	`alertas`	Guarda 0 en alertas
8	`ret`	–	–	–	Termina el bloque

7.2 Bloque gestos (líneas 21 a 29)

Se ejecuta en cada cuadro. Solo calcula las condiciones: el tiempo lo cuenta el autómata de cada gesto (7.6). Por eso no hay un temporizador en las cuádruplas, y una cláusula como `tolerando 100 ms` no genera ninguna.

#	Operador	Arg1	Arg2	Resultado	Qué se hace
1	`<`	`ear`	`umbral_ojo`	`t1`	t1 es verdadero si ear < umbral_ojo
2	`<`	`mar`	`umbral_bostezo`	`t2`	t2 es verdadero si mar < umbral_bostezo
3	`y`	`t1`	`t2`	`t3`	t3 es verdadero si t1 y t2 lo son
4	`afd`	`t3`	–	`parpadeo`	Entrega al AFD de parpadeo el símbolo 1 o 0 del cuadro
5	`<`	`ear`	`umbral_ojo`	`t4`	t4 es verdadero si ear < umbral_ojo
6	`<`	`mar`	`umbral_bostezo`	`t5`	t5 es verdadero si mar < umbral_bostezo
7	`y`	`t4`	`t5`	`t6`	t6 es verdadero si t4 y t5 lo son
8	`afd`	`t6`	–	`microsueno`	Entrega al AFD de microsueno el símbolo 1 o 0 del cuadro
9	`>`	`mar`	`umbral_bostezo`	`t7`	t7 es verdadero si mar > umbral_bostezo
10	`afd`	`t7`	–	`bostezo`	Entrega al AFD de bostezo el símbolo 1 o 0 del cuadro
11	`>`	`pitch`	`cabeceo_grados`	`t8`	t8 es verdadero si pitch > cabeceo_grados
12	`<`	`ear`	`0.27`	`t9`	t9 es verdadero si ear < 0.27
13	`y`	`t8`	`t9`	`t10`	t10 es verdadero si t8 y t9 lo son

#	Operador	Arg1	Arg2	Resultado	Qué se hace
14	afd	t10	–	cabeceo	Entrega al AFD de cabeceo el símbolo 1 o 0 del cuadro
15	>	yaw	giro_grados	t11	t11 es verdadero si yaw > giro_grados
16	neg	giro_grados	–	t12	t12 es giro_grados con el signo cambiado
17	<	yaw	t12	t13	t13 es verdadero si yaw < t12
18	o	t11	t13	t14	t14 es verdadero si t11 o t13 lo es
19	afd	t14	–	distraccion	Entrega al AFD de distraccion el símbolo 1 o 0 del cuadro
20	<	pitch	10	t15	t15 es verdadero si pitch < 10
21	<	yaw	20	t16	t16 es verdadero si yaw < 20
22	y	t15	t16	t17	t17 es verdadero si t15 y t16 lo son
23	neg	20	–	t18	t18 es 20 con el signo cambiado
24	>	yaw	t18	t19	t19 es verdadero si yaw > t18
25	y	t17	t19	t20	t20 es verdadero si t17 y t19 lo son
26	afd	t20	–	atento	Entrega al AFD de atento el símbolo 1 o 0 del cuadro
27	sec	–	–	fatiga	Entrega al AFD de fatiga los eventos del cuadro
28	ret	–	–	–	Termina el bloque

7.3 Bloque al_entrar del estado alerta (líneas 74 a 79)

#	Operador	Arg1	Arg2	Resultado	Qué se hace
1	+	alertas	1	t1	Calcula alertas + 1 y lo guarda en t1
2	=	t1	–	alertas	Guarda t1 en alertas
3	param	"¡Despierte! Riesgo de microsueño"	–	–	Prepara el argumento
4	call	alarma	1	–	Llama a alarma con 1 argumento(s)
5	texto	alertas	int	t2	Convierte alertas (int) a texto
6	concat	"alerta #"	t2	t3	Une los dos textos
7	param	t3	–	–	Prepara el argumento
8	call	registrar	1	–	Llama a registrar con 1 argumento(s)
9	>=	alertas	3	t4	t4 es verdadero si alertas >= 3
10	gotoF	t4	–	13	Si t4 es falso, salta a la cuádrupla 13
11	param	"Se recomienda detener el vehículo"	–	–	Prepara el argumento
12	call	registrar	1	–	Llama a registrar con 1 argumento(s)
13	ret	–	–	–	Termina el bloque

7.4 Bloques mientras (líneas 45 y 59)

Un mientras no tiene salto de regreso. El siguiente cuadro vuelve a ejecutar el bloque desde el inicio, así que nunca detiene el procesamiento del video.

Bloque mientras del estado normal

#	Operador	Arg1	Arg2	Resultado	Qué se hace
1	`>`	`perclos`	`umbral_perclos`	`t1`	t1 es verdadero si perclos > umbral_perclos
2	`gotoF`	`t1`	`–`	`8`	Si t1 es falso, salta a la cuádrupla 8
3	`texto`	`perclos`	`proporcion`	`t2`	Convierte perclos (proporcion) a texto
4	`concat`	`"PERCLOS alto: "`	`t2`	`t3`	Une los dos textos
5	`param`	`t3`	`–`	`–`	Prepara el argumento
6	`call`	`registrar`	`1`	`–`	Llama a registrar con 1 argumento(s)
7	`ir_a`	`–`	`–`	`somnoliento`	Cambia al estado somnoliento
8	`ret`	`–`	`–`	`–`	Termina el bloque

Bloque mientras del estado somnoliento

#	Operador	Arg1	Arg2	Resultado	Qué se hace
1	`call`	`tiempo_en_estado`	`0`	`t1`	Llama a tiempo_en_estado con 0 argumento(s)
2	`>`	`t1`	`15000`	`t2`	t2 es verdadero si t1 > 15000
3	`<`	`perclos`	`umbral_perclos`	`t3`	t3 es verdadero si perclos < umbral_perclos
4	`y`	`t2`	`t3`	`t4`	t4 es verdadero si t2 y t3 lo son
5	`gotoF`	`t4`	`–`	`7`	Si t4 es falso, salta a la cuádrupla 7
6	`ir_a`	`–`	`–`	`normal`	Cambia al estado normal
7	`ret`	`–`	`–`	`–`	Termina el bloque

7.5 Un ciclo repetir

El Listado 1 no usa `repetir`. Este bloque es el `al_entrar` del estado terminado de `ejemplos/rehabilitacion.emo`, que contiene `repetir 3 veces`. El contador `c1` solo existe en este nivel.

#	Operador	Arg1	Arg2	Resultado	Qué se hace
1	`param`	`"¡Ejercicio completo!"`	–	–	Prepara el argumento
2	`call`	`mostrar`	`1`	–	Llama a mostrar con 1 argumento(s)
3	`=`	`0`	–	`c1`	Guarda 0 en c1
4	`<`	`c1`	`3`	`t1`	t1 es verdadero si c1 < 3
5	`gotoF`	`t1`	–	`10`	Si t1 es falso, salta a la cuádrupla 10
6	`param`	`"fin de sesión"`	–	–	Prepara el argumento
7	`call`	`registrar`	`1`	–	Llama a registrar con 1 argumento(s)
8	`+`	`c1`	`1`	`c1`	Calcula c1 + 1 y lo guarda en c1
9	`goto`	–	–	`4`	Salta a la cuádrupla 4
10	`ret`	–	–	–	Termina el bloque

7.6 Relación con los autómatas

El anteproyecto pide traducir gestos y estados a autómatas finitos con tabla de transiciones y diagrama. Las cuádruplas `afd` y `sec` son la unión entre ambas partes. Un gesto se compila a un AFD sobre el alfabeto {1, 0}, donde `q_i` significa que la condición lleva i cuadros. La duración se discretiza con el fps: `N = techo(durante / periodo)`. Las secuencias usan un AFD construido con el algoritmo KMP y las máquinas de estado se traducen directamente, con λ para las transiciones espontáneas. `python -m emolang compilar` genera todas las tablas y los diagramas.

Figura 2. AFD de un gesto de tres cuadros; la transición marcada emite el evento g

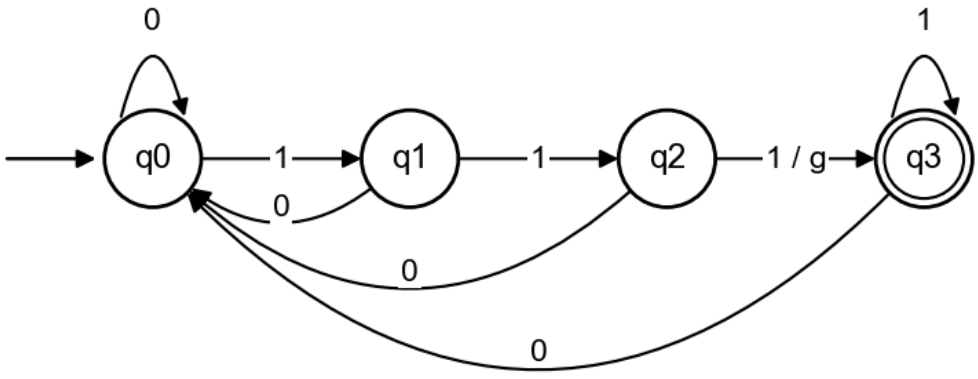


Tabla de transiciones de la secuencia fatiga (→ inicial, * aceptación, / evento emitido)

δ	bostezo	cabeceo
→ s0	s1	s0
s1	s1	s2 / fatiga
* s2	s1	s0

Transiciones de la máquina monitor (λ = espontánea)

Estado	Evento	Destino	Condición
normal	parpadeo	normal	
normal	bostezo	somnoliento	
normal	microsueno	alerta	
normal	distraccion	distraido	
normal	fatiga	alerta	
normal	λ	somnoliento	perclos > umbral_perclos
somnoliento	microsueno	alerta	
somnoliento	fatiga	alerta	
somnoliento	distraccion	distraido	
somnoliento	cabeceo	somnoliento	
somnoliento	λ	normal	tiempo_en_estado() > 15 s y perclos < umbral_perclos
distraido	atento	normal	
distraido	microsueno	alerta	
distraido	tras 30 s	alerta	
alerta	atento	normal	

8. Técnicas de optimización

El optimizador aplica seis técnicas sobre las cuádruplas de la sección 7, hasta que ya ninguna produce cambios. El Listado 1 baja de 83 a 68 cuádruplas y `ejemplos/rehabilitacion.emo` de 63 a 54. Las reutilizaciones de subexpresiones se limitan a un mismo bloque, porque los temporales no se conservan entre bloques.

8.1 Técnicas

Técnica	Qué hace	Ejemplo real	Veces
Propagación de constantes	Cambia una variable con una sola escritura constante por su valor	`(<, ear, umbral_ojo, t1)` queda `(<, ear, 0.2, t1)`	10
Plegado de constantes	Calcula al compilar las operaciones que solo tienen constantes	`(neg, 20, –, t12)` queda `(=, -20, –, t12)`	2
Eliminación de subexpresiones comunes	Reutiliza una operación ya calculada en el bloque	`(<, ear, 0.2, t4)`: se reutiliza t1	3
Eliminación de código muerto	Quita lo que no afecta al resultado: escrituras que nadie lee, código inalcanzable, gestos y secuencias sin uso	`(=, 0, –, normal.parpadeos)`: su resultado nunca se lee	10
Mirilla	Une una operación con la asignación que la sigue	`(+, alertas, 1, t1)` y `(=, t1, –, alertas)` queda `(+, alertas, 1, alertas)`	2
Desenrollado de ciclos	Cambia un repetir constante de hasta 4 vueltas por copias de su cuerpo	repetir 3 veces (7 cuádruplas con contador) queda en 6 cuádruplas sin saltos (rehabilitación)	1

La columna Veces cuenta las aplicaciones en el Listado 1; el desenrollado, en rehabilitación. La variable `parpadeos` es un caso que vale la pena revisar: se incrementa, pero nada la lee, así que el optimizador quita esa suma y el bloque `normal.cuando.parpadeo` queda vacío. El evento sigue detectándose. Un gesto sin uso también se elimina al ejecutar, salvo en `emolang evaluar`, que necesita medir todos los gestos declarados.

8.2 Resultado por bloque

Bloque	Antes	Después
`init`	8	2
`gestos`	28	23
`normal.al_entrar`	3	2
`normal.cuando.parpadeo`	3	1
`normal.cuando.bostezo`	4	4
`normal.mientras.1`	8	8
`somnoliento.al_entrar`	3	3
`somnoliento.cuando.cabeceo`	3	3
`somnoliento.mientras.1`	7	7
`distraido.al_entrar`	3	3
`alerta.al_entrar`	13	12
`Total`	83	68

8.3 Bloque gestos optimizado

#	Operador	Arg1	Arg2	Resultado	Qué se hace
1	<	ear	0.2	t1	t1 es verdadero si ear < 0.2
2	<	mar	0.33	t2	t2 es verdadero si mar < 0.33
3	y	t1	t2	t3	t3 es verdadero si t1 y t2 lo son
4	afd	t3	–	parpadeo	Entrega al AFD de parpadeo el símbolo 1 o 0 del cuadro
5	afd	t3	–	microsueno	Entrega al AFD de microsueno el símbolo 1 o 0 del cuadro
6	>	mar	0.33	t7	t7 es verdadero si mar > 0.33
7	afd	t7	–	bostezo	Entrega al AFD de bostezo el símbolo 1 o 0 del cuadro
8	>	pitch	10	t8	t8 es verdadero si pitch > 10
9	<	ear	0.27	t9	t9 es verdadero si ear < 0.27
10	y	t8	t9	t10	t10 es verdadero si t8 y t9 lo son
11	afd	t10	–	cabeceo	Entrega al AFD de cabeceo el símbolo 1 o 0 del cuadro
12	>	yaw	20	t11	t11 es verdadero si yaw > 20
13	<	yaw	-20	t13	t13 es verdadero si yaw < -20
14	o	t11	t13	t14	t14 es verdadero si t11 o t13 lo es
15	afd	t14	–	distraccion	Entrega al AFD de distraccion el símbolo 1 o 0 del cuadro
16	<	pitch	10	t15	t15 es verdadero si pitch < 10
17	<	yaw	20	t16	t16 es verdadero si yaw < 20
18	y	t15	t16	t17	t17 es verdadero si t15 y t16 lo son
19	>	yaw	-20	t19	t19 es verdadero si yaw > -20
20	y	t17	t19	t20	t20 es verdadero si t17 y t19 lo son
21	afd	t20	–	atento	Entrega al AFD de atento el símbolo 1 o 0 del cuadro
22	sec	–	–	fatiga	Entrega al AFD de fatiga los eventos del cuadro
23	ret	–	–	–	Termina el bloque

8.4 Bloque al_entrar de alerta optimizado

#	Operador	Arg1	Arg2	Resultado	Qué se hace
1	`+`	`alertas`	`1`	`alertas`	Calcula alertas + 1 y lo guarda en alertas
2	`param`	`"¡Despierte! Riesgo de microsueño"`	–	–	Prepara el argumento
3	`call`	`alarma`	`1`	–	Llama a alarma con 1 argumento(s)
4	`texto`	`alertas`	`int`	`t2`	Convierte alertas (int) a texto
5	`concat`	`"alerta #"`	`t2`	`t3`	Une los dos textos
6	`param`	`t3`	–	–	Prepara el argumento
7	`call`	`registrar`	`1`	–	Llama a registrar con 1 argumento(s)
8	`>=`	`alertas`	`3`	`t4`	t4 es verdadero si alertas >= 3
9	`gotoF`	`t4`	–	`12`	Si t4 es falso, salta a la cuádrupla 12
10	`param`	`"Se recomienda detener el vehículo"`	–	–	Prepara el argumento
11	`call`	`registrar`	`1`	–	Llama a registrar con 1 argumento(s)
12	`ret`	–	–	–	Termina el bloque

8.5 Ciclo repetir desenrollado

El `repetir 3 veces` del bloque de 7.5 ejecutaba 21 cuádruplas del ciclo: la inicialización, seis por vuelta y las dos de la última comparación. Ahora ejecuta seis, sin contador ni saltos.

#	Operador	Arg1	Arg2	Resultado	Qué se hace
1	`param`	`"¡Ejercicio completo!"`	–	–	Prepara el argumento
2	`call`	`mostrar`	1	–	Llama a mostrar con 1 argumento(s)
3	`param`	`"fin de sesión"`	–	–	Prepara el argumento
4	`call`	`registrar`	1	–	Llama a registrar con 1 argumento(s)
5	`param`	`"fin de sesión"`	–	–	Prepara el argumento
6	`call`	`registrar`	1	–	Llama a registrar con 1 argumento(s)
7	`param`	`"fin de sesión"`	–	–	Prepara el argumento
8	`call`	`registrar`	1	–	Llama a registrar con 1 argumento(s)
9	`ret`	–	–	–	Termina el bloque

8.6 Comprobación

Una optimización no debe cambiar lo que hace el programa. El proyecto tiene 64 pruebas automáticas; las de equivalencia ejecutan los dos programas de ejemplo en 30 corridas simuladas, con 5 semillas, ruido normal y triple. En cada corrida comparan el intérprete directo del árbol, EmoVM sin optimizar y EmoVM optimizada. Las tres deben producir la misma bitácora, la misma salida y los mismos estados en cada cuadro.

9. Plataforma y arquitectura

EmoLang compila a un archivo `.emoc` con los autómatas, la tabla de despacho de los estados y las cuádruplas. Ese archivo lo ejecuta una máquina virtual propia, EmoVM, escrita en Python, en una computadora con CPU y cámara web.

9.1 Elección de la plataforma

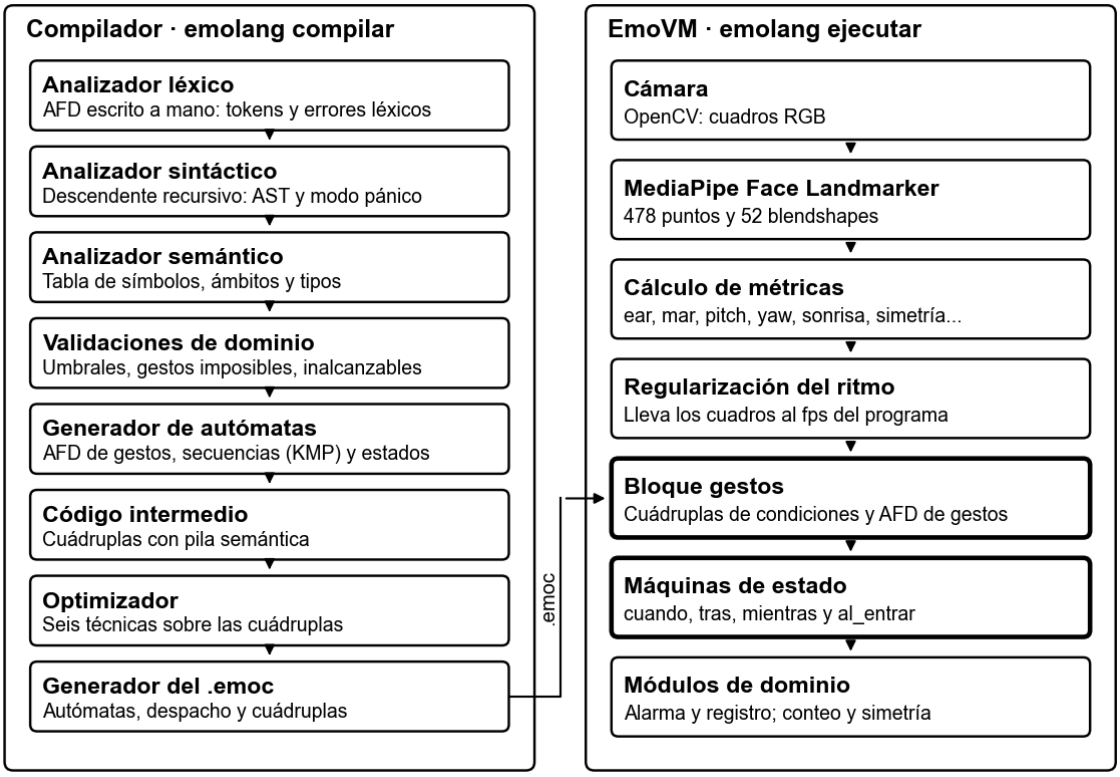
Opción	A favor	En contra	Decisión
Código nativo x86-64	Mayor rendimiento	Habría que conectar MediaPipe y la cámara desde C/C++; no cabe en el semestre	Descartada
Traducir EmoLang a Python	Rápido de construir	Las cuádruplas y los autómatas ya no se pueden mostrar ni revisar	Descartada
Máquina virtual propia (EmoVM)	Se integra directo con MediaPipe; las cuádruplas y los autómatas se pueden mostrar y probar	Más lenta que el código nativo, pero suficiente: ver abajo	Elegida

En la prueba simulada del Listado 1, EmoVM tardó en promedio 0.028 ms por cuadro (p95 de 0.043 ms), de los 33.3 ms disponibles a 30 fps. Esa medición no incluye la extracción de puntos faciales de MediaPipe. En las pruebas con la cámara del equipo se obtuvieron unos 10 fps, y el

sistema regulariza el ritmo para que `durante 1 s` siga significando un segundo.

9.2 Arquitectura

Figura 3. Arquitectura del compilador y de la máquina virtual



La columna izquierda se ejecuta una vez, al compilar. La derecha se ejecuta en cada cuadro de video. El archivo `.emoc` que escribe el generador es lo que carga la máquina virtual.

9.3 Tecnologías

Componente	Tecnología
Lenguaje de implementación	Python 3.11 con cámara (MediaPipe no soporta 3.13 o posterior); el compilador funciona desde Python 3.10
Análisis léxico y sintáctico	Escritos a mano: AFD y descendente recursivo, sin generador
Captura de video	OpenCV
Puntos faciales	MediaPipe Face Landmarker
Cálculo de métricas	NumPy
Diagrama de los autómatas	Mermaid y archivos DOT de Graphviz, generados por el compilador
Sonido de la alarma	winsound de Python en Windows; aviso de la terminal en otros sistemas
Equipo	Laptop x86-64 sin GPU dedicada, con cámara web

El analizador se escribió a mano y no con PLY u otro generador porque así cada fase se puede revisar y explicar directamente, se controlan los mensajes de error y la recuperación en modo pánico, y el compilador no depende de otra biblioteca. Una gramática LL(1) como esta se presta bien a ese método.

Referencias

- Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.ª ed.). Pearson Educación.
- Knuth, D. E., Morris, J. H. y Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2), 323-350.
- Louden, K. C. (2004). *Construcción de compiladores: principios y práctica*. Thomson.
- Lugaresi, C. et al. (2019). MediaPipe: A framework for building perception pipelines. arXiv:1906.08172.
- Soukupová, T. y Čech, J. (2016). Real-time eye blink detection using facial landmarks. *21st Computer Vision Winter Workshop*.
- Wierwille, W. W. et al. (1994). *Research on vehicle-based driver status/performance monitoring*. Reporte DOT HS 808 247, NHTSA.
- Google. MediaPipe Face Landmarker. Documentación oficial de MediaPipe Solutions.